UNUM OS: CLAIMS & POLICY ARCHITECTURE

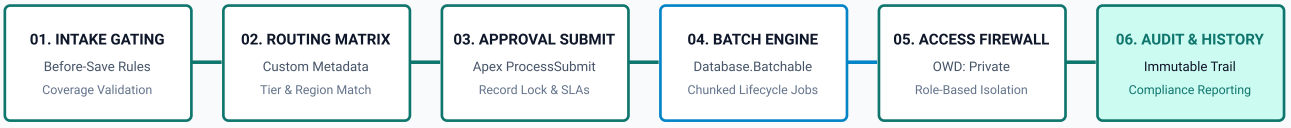
ASYNCHRONOUS POLICY ENGINE, DYNAMIC GOVERNANCE & REGULATORY COMPLIANCE

Service Cloud • Insurance Engine

Database.Batchable | Custom Metadata | OWD: Private

PLATFORM BOUNDARIES & END-TO-END ORCHESTRATION FLOW

Salesforce Service Cloud acts as the central claims adjudication and policy servicing platform. The architecture separates high-velocity customer claims operations, off-peak bulk policy maintenance, and regulatory data governance into decoupled execution layers.



CORE ENGINEERING PILLARS & ARCHITECTURAL STRENGTHS

1. Scalable Asynchronous Architecture

Database.Batchable

Bulk policy lifecycle management and recurring status re-calculations require processing without hitting the 50,000 SOQL row governor limit or causing heap collapses.

- **Query Locator:** Implemented `Database.getQueryLocator` to stream millions of policy records safely.
- **Stateful Logging:** Used `Database.Stateful` to aggregate partial DML failures into an operational error table without rolling back the entire execution batch.

2. Dynamic Multi-Tier Approvals

Custom Metadata Driven

Replaced rigid declarative approval setups with metadata-configured routing matrices that evaluate payout tiers, insurance line, and regional jurisdiction.

- **Decoupled Logic:** Thresholds configured in `Claim_Routing_Rule__mdt` eliminate code deployments for threshold updates.
- **Concurrency Protection:** Programmatic submission automatically locks the claim record, preventing race conditions between concurrent adjusters.

3. Regulatory Data Security & Isolation

OWD Private + Granular Sharing

Enforced strict regulatory compliance and Separation of Duties (SoD) across sensitive financial and healthcare insurance policies.

- **Access Isolation:** Baseline `OWD: Private` with territorial Role Hierarchies guarantees zero cross-region claim visibility.
- **Permission Set Groups:** Separated operational privileges (Field Adjuster vs Senior Claims Auditor) cleanly from baseline profiles.

4. Enterprise Data Ingestion Framework

High-Fidelity Migration

Engineered idempotent ingestion pipelines for legacy policy records, maintaining audit history while suppressing transactional side-effects.

- **Automation Bypass Pattern:** Hierarchical bypass flags temporarily silenced email alerts and validation rules during mass ingestion.
- **Indexed Upserting:** Keyed against external identifiers to avoid duplicate creation and preserve historical policy integrity.

BACKEND ENGINEERING: BATCH & APPROVALS

BATCH APEX OPTIMIZATION, CONCURRENCY CONTROL & PROGRAMMATIC APPROVALS

Backend Engineering

1. Enterprise Batch Apex Framework (PolicyLifecycleBatch)

Database.Batchable • Database.Stateful

Architecture Pattern: High-volume policy lifecycle transitions (expiration checks, premium arrears evaluation, and status recalculation) run via chunked Batch Apex with isolated error logging.

```
public class PolicyLifecycleBatch implements Database.Batchable<sObject>, Database.Stateful { public Integer totalProcessed = 0; public List<String> failureLogs = new List<String>(); public Database.QueryLocator start(Database.BatchableContext bc) { return Database.getQueryLocator([ SELECT Id, Status__c, Expiration_Date__c FROM Policy__c WHERE Is_Active__c = true ]); } public void execute(Database.BatchableContext bc, List<Policy__c> scope) { for (Policy__c pol : scope) { if (pol.Expiration_Date__c <= Date.today()) { pol.Status__c = 'Expired'; } } Database.SaveResult[] srList = Database.update(scope, false); // Aggregate partial DML failures via Database.Stateful } public void finish(Database.BatchableContext bc) { // Dispatches operational summary email or inserts error audit log } }
```

Execution & Governor Controls:

1. **QueryLocator vs Iterable:** QueryLocator bypasses the 50,000 SOQL row ceiling, querying up to 50M records safely across cursor batches.
2. **Chunk Sizing (200 records):** Balances CPU timeout (10,000ms) and avoids hitting apex heap size limit (12MB async).
3. **Partial Commit (`allOrNone = false`):** Ensures failed row locks or validation errors do not abort the entire 200-record batch.
4. **Deadlock Mitigation:** Batch queries sort by Parent Account ID to eliminate cross-transaction row locking on parent records.

2. Programmatic Multi-Tier Approval Dispatch Engine

Approval.ProcessSubmitRequest • Custom Metadata

Architecture Pattern: Dynamic routing matrices evaluate claim payout thresholds and region to route approvals programmatically via Apex, decoupling routing definitions from hardcoded User IDs.

```
public static void submitClaimForApproval(Claim__c claimRec) { // 1. Resolve Approver ID dynamically from Custom Metadata Id targetApproverId = ClaimRoutingService.resolveApprover( claimRec.Claim_Amount__c, claimRec.Policy_Type__c, claimRec.Region__c ); // 2. Programmatic approval request invocation Approval.ProcessSubmitRequest req = new Approval.ProcessSubmitRequest(); req.setComments('Automated triage submit based on metadata matrix.'); req.setObjectId(claimRec.Id); req.setNextApproverIds(new Id[] { targetApproverId }); req.setProcessDefinitionNameOrId('Tiered_Claim_Approval_v2'); req.setSkipEntryCriteria(true); Approval.ProcessResult result = Approval.process(req); }
```

Governance & Concurrency Advantages:

A. **Metadata Matrix:** Routing thresholds (e.g., <$25k Supervisor, $25k–$100k Regional Mgr, >$100k VP) update via admin records without code deployment.

B. **Automatic Record Lock:** System locks the claim record immediately upon submit, eliminating edit conflicts between concurrent claims adjusters.

C. **Compliance Audit Trail:** Every routing action generates native `ProcessInstance` and `ProcessInstanceStep` history records for auditing.

Architectural Takeaway: Mitigating Row-Locking (UNABLE_TO_LOCK_ROW)

When high-frequency adjusters edit claims during daytime operations while scheduled batch jobs recalculate policy statuses, database lock contention occurs. We mitigated this by scheduling bulk batch jobs during off-peak processing windows (2:00 AM) and ordering query batches by Parent Account ID to eliminate cross-transaction locking.

SECURITY, MIGRATION & GOVERNANCE MATRIX

ROLE-BASED ISOLATION, INGESTION BYPASS PATTERNS & MASTER REFERENCE

Compliance Dossier

Regulatory Data Security Firewall

OWD: Private

Protected sensitive policy and financial claims data across segregated business units.

1. **OWD Private Baseline:** All Policy and Claim records restricted from baseline organizational exposure.
2. **Territorial Hierarchy:** Role hierarchy aligned to operational regions (East, West, Central) enforces geographic isolation.
3. **Criteria-Based Sharing:** Selectively shares high-exposure claims (>$100k) with Special Investigations and Legal Compliance.
4. **Permission Set Groups:** Decoupled privilege management (e.g., Senior Auditor, Claims Adjuster) without modifying base profiles.

Enterprise Ingestion & Bypass Pattern

Bulk Upsert Architecture

High-integrity legacy ingestion pipeline preserving historical audit trails without side-effects.

1. **Indexed External IDs:** Executed idempotent upserts against indexed `Legacy_Policy_Number__c`.
2. **Automation Bypass Flag:** Hierarchical Custom Settings suppressed validation rules and triggers during mass ingestion passes.
3. **Suppression of Outbound Alerts:** Prevented false email notifications and task creation for historical records.
4. **Pre-Load Deduplication:** Standardized tax IDs and policy numbers before ingestion, maintaining zero record loss.

COMPREHENSIVE ARCHITECTURE & GOVERNANCE MATRIX

Architecture Component	Implementation Pattern	Governor & Concurrency Guardrail	Enterprise Business Impact
Policy Lifecycle Engine	Database.Batchable + Database.Stateful	QueryLocator 50M row limit; chunk size 200	Automated bulk status updates and maintenance
Dynamic Claim Routing	Custom Metadata Types + Apex	Cached SOQL query; zero DML in routing evaluation	Decoupled approver assignment without code redeployment
Programmatic Approvals	Approval.ProcessSubmitRequest	Automated record-lock prevents race conditions	Audit-grade approval tracking and SLA enforcement
Access Control Firewall	OWD Private + Criteria-Based Sharing	Optimized sharing groups prevent lock contention	Enforces strict regulatory compliance and regional SoD
Legacy Data Ingestion	Bulk API Upsert via Indexed External ID	Idempotent upserting prevents duplicate row generation	High-fidelity historical data ingestion with zero loss
Automation Bypass Flag	Hierarchical Custom Settings / Permissions	Zero-cost boolean evaluation at trigger entry	Prevents false alert floods and validation blocks during loads
Claims Service Console	Service Cloud Case & Claim Integration	Standard console workspace integration	Centralized operational visibility; reduced handling time

Interview Executive Summary Note

Position the UNUM OS project as a robust enterprise insurance engine that balances scalable asynchronous data processing (`Database.Batchable`), metadata-driven approval routing, and regulatory data isolation (`OWD: Private`). Highlight your understanding of governor limits, partial commit error handling, and row-lock mitigation.